

CSE 331

AUTOMATA AND COMPUTABILITY

Lecture 2

Spring 2026

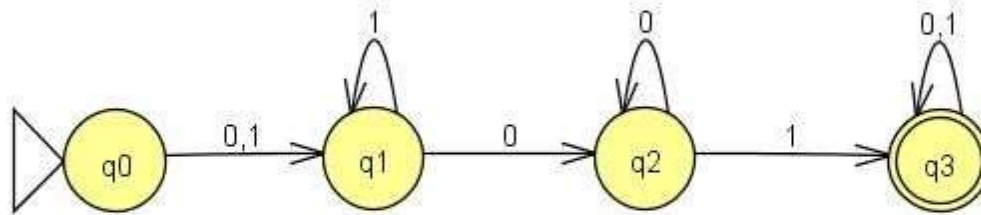
Today's learning goals

Sipser Ch 1.1

- Use and design a finite automaton via its
 - Formal definition
 - state diagram
- Identify the strings and languages accepted by a given finite automaton
- Design a finite automaton which accepts a given language
- Define the regular operations on languages

Deterministic Finite Automaton (DFA)

- Input: finite **string** over a fixed **alphabet**
- Output: "accept" or "reject"



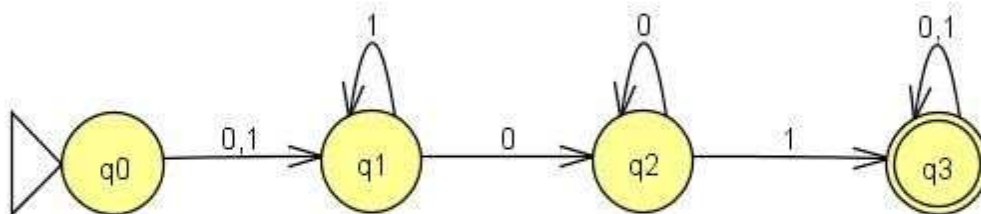
Start state
(triangle/arrow)

Accept state
(double circle)

Language of the machine is the set of strings it accepts

Deterministic finite automaton

- Input: finite string over a fixed alphabet
- Output: "accept" or "reject"



Does this DFA accept the **empty string** ?

- A. Yes
- B. No
- C. I don't know

Deterministic finite automaton

Sipser p. 35 Def 1.5

A **finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where

1. Q is a finite set called the states
2. Σ is a finite set called the alphabet
3. $\delta : Q \times \Sigma \rightarrow Q$ is the transition function
4. $q_0 \in Q$ is the start state
5. $F \subseteq Q$ is the set of accept states.



**No circles and
arrows, same
information!**

Deterministic finite automaton

Sipser p. 35 Def 1.5

A **finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where

1. Q is a finite set called the states
2. Σ is a finite set called the alphabet
3. $\delta : Q \times \Sigma \rightarrow Q$ is the transition function
4. $q_0 \in Q$ is the start state
5. $F \subseteq Q$ is the set of accept states.

Can there be more than one **start state** in a finite automaton?

- A. Yes, because of line 4.
- B. No, because of line 4.
- C. I don't know

Deterministic finite automaton

Sipser p. 35 Def 1.5

A **finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where

1. Q is a finite set called the states
2. Σ is a finite set called the alphabet
3. $\delta : Q \times \Sigma \rightarrow Q$ is the transition function
4. $q_0 \in Q$ is the start state
5. $F \subseteq Q$ is the set of accept states.

How many outgoing arrows from each state?

- A. May be different number at each state.
- B. Must be 2.
- C. Must be $|Q|$.
- D. Must be $|\Sigma|$
- E. I don't know.

An example

Define $M = (\{q1, q2, q3, q4\}, \{a, b\}, \delta, q1, \{q4\})$ where the function δ is specified by its table of values:

Input in $Q \times \Sigma$	Output in Q
(q1,a)	q3
(q2,a)	q2
(q3,a)	q3
(q4,a)	q2

Input in $Q \times \Sigma$	Output in Q
(q1,b)	q2
(q2,b)	q2
(q3,b)	q4
(q4,b)	q4

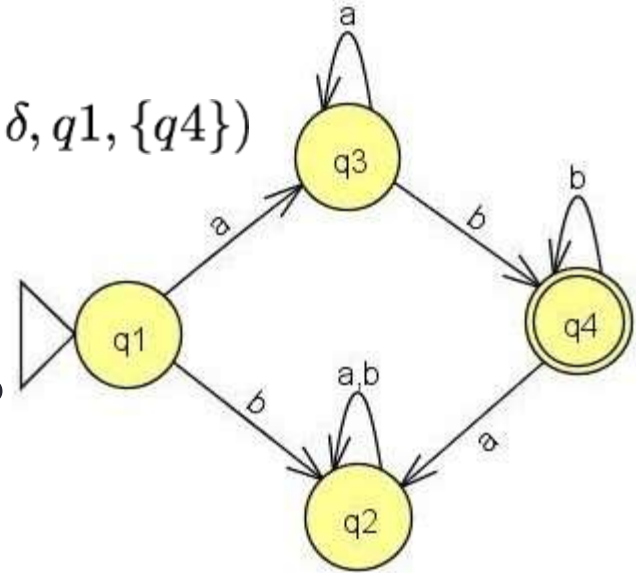
Draw the state diagram for the DFA with this formal definition.

An example

$(\{q1, q2, q3, q4\}, \{a, b\}, \delta, q1, \{q4\})$

What's an example of a

- length 1 string accepted by this DFA?
- length 1 string rejected by this DFA?
- length 2 string accepted by this DFA?
- length 2 string rejected by this DFA?

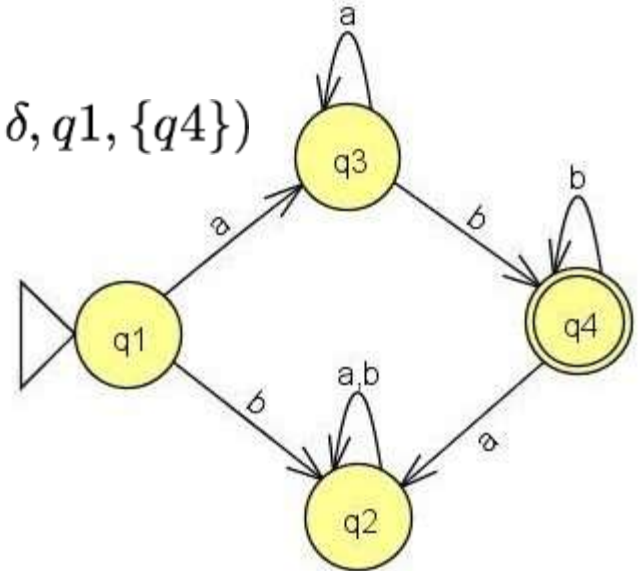


An example

$(\{q1, q2, q3, q4\}, \{a, b\}, \delta, q1, \{q4\})$

What's the best description of the language recognized by this DFA?

- A. Starts with b and ends with a or b
- B. Starts with a and ends with a or b
- C. a's followed by b's
- D. More than one of the above
- E. I don't know.



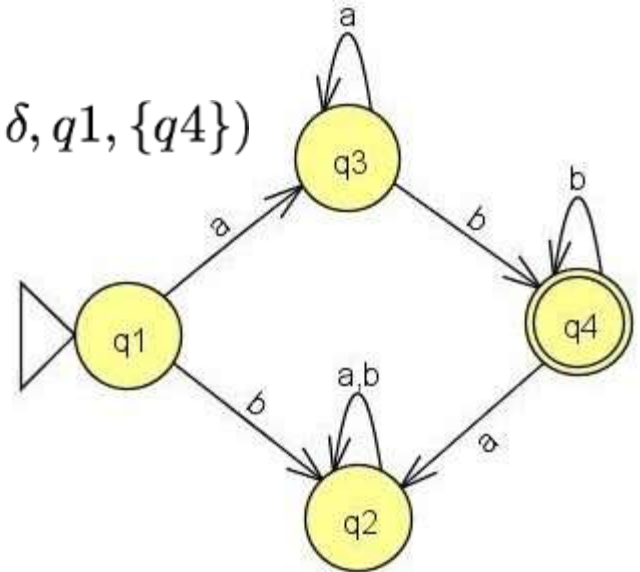
and using set notation?

An example

$(\{q1, q2, q3, q4\}, \{a, b\}, \delta, q1, \{q4\})$

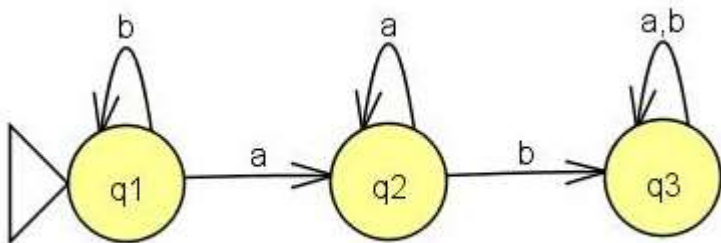
This DFA recognizes
the language of all strings
of the form a's followed by b's

i.e. $\{a^n b^k \mid n, k \geq 1\}$



Specifying an automaton

($\{q_1, q_2, q_3\}$, $\{a, b\}$, δ , q_1 , ?)

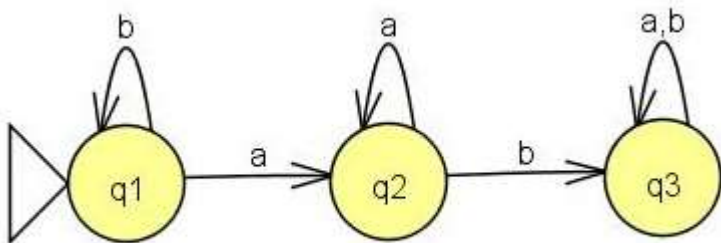


What state(s) should be in F so that the language of this machine is
 $\{ w \mid ab \text{ is a substring of } w \}$?

- A. $\{q_2\}$
- B. $\{q_3\}$
- C. $\{q_1, q_2\}$
- D. $\{q_1, q_3\}$
- E. I don't know.

Specifying an automaton

($\{q_1, q_2, q_3\}$, $\{a, b\}$, δ , q_1 , ?)



What state(s) should be in F so that the language of this machine is
 $\{ w \mid \text{b's never occur after a's in } w \}$?

- A. $\{q_2\}$
- B. $\{q_3\}$
- C. $\{q_1, q_2\}$
- D. $\{q_1, q_3\}$
- E. I don't know.

Building DFA



Typical questions

Define a DFA which recognizes the given language L .

or

Prove that the (given) language L is regular.

Building DFA



Example

Define a DFA which recognizes

$$L_1 = \{w \mid w \text{ ends with a } 1\}$$

Building DFA



Example

Define a DFA which recognizes

$$L_2 = \{w \mid w \text{ has substring } 11\}$$

Building DFA



Example

Define a DFA which recognizes

$$L_3 = \{w \mid w \text{ has even length}\}$$

Building DFA



Example

Define a DFA which recognizes

$$L_4 = \{w \mid \text{length of } w \text{ is two}\}$$

Building DFA



Example

Define a DFA which recognizes

$$L_5 = \{w \mid w \text{ starts with } 1 \text{ and ends with } 0\}$$

Building DFA



Example

Define a DFA which recognizes

$$L_6 = \{w \mid w \text{ has at least 2 a's}\}$$

Building DFA



Example

Define a DFA which recognizes

$$L_7 = \{w \mid w \text{ has at most 2 a's}\}$$

Building DFA



Example

Define a DFA which recognizes

$L_8 = \{w \mid w \text{ starts and ends with a or } w \text{ starts and ends with } b\}$



Building DFA



Remember

States are our only (computer) memory.

Design and pick states with specific roles / tasks in mind.

"Have not seen any of desired pattern yet"

"Trap state"

Justification?

To prove that the DFA we build, M , actually recognizes the language L

$$\text{WTS } L(M) = L$$

(1) Is every string accepted by M in L ?

(2) Is every string from L accepted by M ?

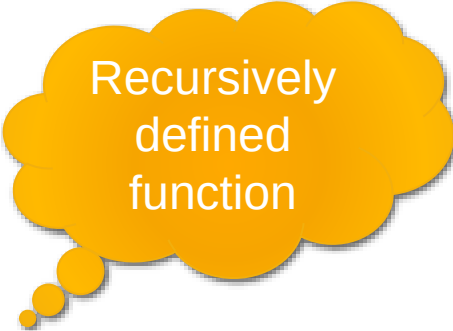
or contrapositive version: Is every string rejected by M not in L .

A useful (optional) bit of terminology

When is a string accepted by a DFA?

Computation of M on w: *where do we land when start at q_0 and read each symbol of w one-at-a time?*

$$\delta^*(q, w) =$$



Recursively
defined
function

Regular languages

Sipser p. 35 Def 1.5

- For DFA M over the alphabet Σ
 - For each string w over Σ , M either accepts w or rejects w
 - The **language recognized by M** is the set of strings M accepts
 - The **language of M** is the set of strings M accepts
 - **$L(M)$** = $\{ w \mid w \text{ is a string over } \Sigma \text{ and } M \text{ accepts } w \}$

Definition: A language is **regular** iff there is some finite automaton that recognizes **exactly** it.

Regular languages: bounds?

Is **every** finite language regular?

- A. No: some finite languages are regular, and some are not.
- B. No: there are no finite regular languages.
- C. Yes: every finite language is regular.
- D. I don't know.

For next time

- Read the assigned reading
- Start solving the DFA portion of the assignment that will be released soon.
- *Review Sipser Ch 0 as needed.*